

KIO1 - KIO2 Integration Strategy and Communication Protocol

Bug Locate & Fix: Reverse Execution & Dynamic Slicing Component

Field	Value
Document type	Technical integration specification
Responsible partner	BitNet @Yasin Ünal, @Hemant Singh
Scope	KIO1 - KIO2 integration schema
Version	0.3 (draft for discussion)
Date	25 August 2026
Related documents	KIO1 - KIO10 Integration Strategy, KIO1 - KIO9 Integration Protocol v0.4; KIO1 - KIO12 Integration Protocol v0.5

Table of Contents

...

Table of Abbr.

...

Version History

Version	Description
0.2 = > 0.3	Version 0.3 keeps every statement of version 0.2 and adds the parts of the protocol that were left implicit. The additions are the transport rules that apply to every message, the vocabulary for reporting a failure, the handling of analyses that do not fit in a single synchronous exchange, the recorded values a reply carries and how long they are kept, the configuration and deployment profile of the service, a summary of how each

	capability is used in a workflow, and the compatibility rules for the versions that follow. Two of these describe behaviour that is proposed rather than implemented: asynchronous processing of long analyses, and the closed set of error codes. Both are marked where they appear, and neither changes what a version 0.2 caller sees today. The section order follows the KIO1 to KIO9 and KIO1 to KIO12 protocol documents, so that a partner integrating several agents meets the same subjects in the same places.
--	---

1. KIO1 – KIO2 Integration

Following the approach established in the shared KIO1–KIO10 integration document, the same methodology was applied to report the KIO1–KIO2 interaction, remaining fully compliant with the structure defined for KIO1. This report details the 'step' schema designed for KIO2 and its realization within the execution workflow. Specifically, it covers the integration of KIO2 with KIO1 (Figure 1) and the workflow orchestration component. This provides a limited but representative integration scenario in which workflow planning, step execution, API communication, and result handling can be validated before additional KIOs are introduced.

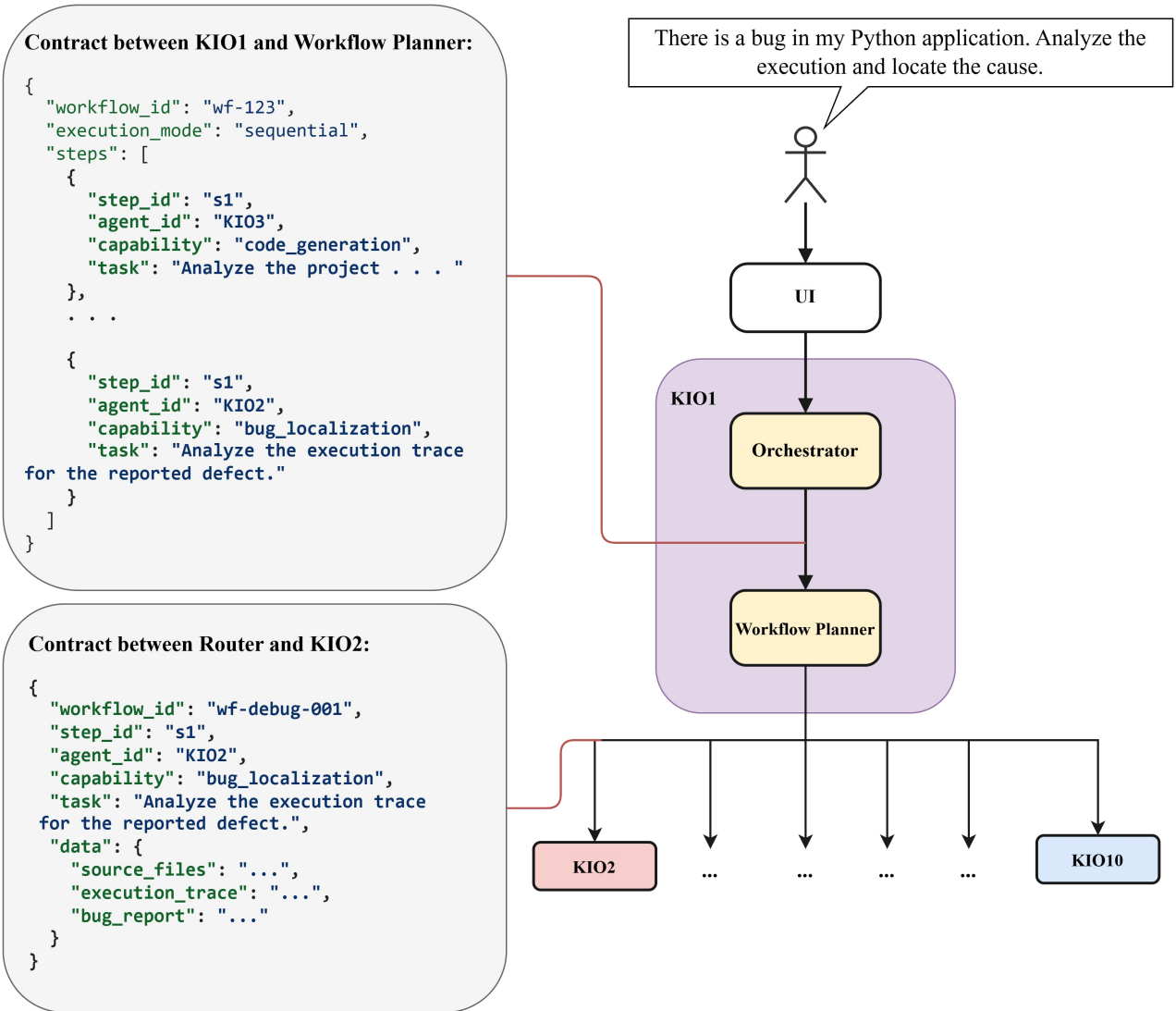


Figure 1. Progressive system integration and communication protocol sample for KIO2

For a given user request, KIO1 first converts the request into a structured workflow. Each workflow step specifies the target agent, required capability, and task. For example, when developing a [medical shift scheduling application \(Use Case 2\)](#), KIO1 may identify an energy-efficiency analysis step for assessing whether selected components can be deployed efficiently on edge or resource-constrained devices. The workflow therefore contains a step such as:

INPUT:

```
>>>Create a FastAPI CRUD service for managing hospital medical shift schedules.
```

RAW OUTPUT:

```
{
  "workflow_id": "wf-fastapi-shifts-001",
  "execution_mode": "sequential",
  "steps": [
    {
      "step_id": "s1",
      "agent_id": "KI03",
      "capability": "requirements_engineering",
      "task": "Define the functional and non-functional requirements for a FastAPI-based medical shift scheduling service, including user roles, shift creation and management, scheduling constraints, and API requirements."
    },
    {
      "step_id": "s2",
      "agent_id": "KI04",
      "capability": "architecture_design",
      "task": "Design the architecture of the FastAPI medical shift scheduling service, including API structure, database integration, authentication, data models, and component interactions."
    },
    {
      "step_id": "s3",
      "agent_id": "KI07",
      "capability": "code_generation",
      "task": "Generate the FastAPI application implementing the defined requirements and architecture, including CRUD endpoints, database models, schemas, and application configuration."
    },
    {
      "step_id": "s4",
      "agent_id": "KI02",
      "capability": "bug_localization",
      "task": "Analyze the generated FastAPI service for potential runtime and logical defects, identify suspicious code locations and execution paths, and provide debugging findings for detected issues."
    },
    {
      "step_id": "s5",
      "agent_id": "KI09",
      "capability": "compliance_review",
      "task": "Review the FastAPI service for applicable privacy, data protection, and responsible AI requirements related to medical shift data."
    },
    {
      "step_id": "s6",
```

```

    "agent_id": "KIO12",
    "capability": "cybersecurity_validation",
    "task": "Validate the security of the FastAPI service, including
authentication, authorization, API exposure, input validation, and protection of
shift-related data."
  },
  {
    "step_id": "s7",
    "agent_id": "KIO11",
    "capability": "test_automation",
    "task": "Create automated tests for the FastAPI CRUD endpoints, scheduling
logic, validation rules, authentication, and error handling."
  },
  {
    "step_id": "s8",
    "agent_id": "KIO10",
    "capability": "energy_efficiency",
    "task": "Analyze the computational and energy-efficiency characteristics of
the application and identify opportunities for efficient deployment on resource-
constrained or edge environments."
  },
  {
    "step_id": "s9",
    "agent_id": "KIO8",
    "capability": "deployment",
    "task": "Package and deploy the FastAPI service with the required
configuration and deployment settings."
  }
],
"explanation": "Full development workflow for a FastAPI-based medical shift
scheduling service, including requirements engineering, architecture design, code
generation, debugging and bug localization, compliance and security validation,
automated testing, energy-efficiency analysis, and deployment."
}

```

The raw workflow output is then passed from KIO1 to the Workflow Planner (Figure 1). The Workflow Planner interprets the workflow and converts each workflow step into a structured execution message for the corresponding KIO. For the KIO2 step, the resulting message is:

```

{
  "workflow_id": "wf-fastapi-shifts-001",
  "step_id": "s4",
  "agent_id": "KIO2",
  "capability": "bug_localization",
  "endpoint": "http://kio2:8102",
  "task": "Analyze the generated FastAPI service for potential runtime and logical
defects, identify suspicious code locations and execution paths, and provide
debugging findings for detected issues.",
  "data": {
    "source_artifact": "fastapi-shift-service",
    "source_location": "...",

```

```

    "execution_trace": null,
    "bug_report": null
  }
}

```

This message represents the communication contract between the Workflow Planner and KIO2 (Figure 1). The *workflow_id* and *step_id* identify the workflow and the specific task, while *agent_id* and *capability* identify the target KIO and requested capability. The task field provides the natural-language instruction, while the data field provides additional inputs required by KIO2 to perform the debugging and bug-localization task. Example capabilities are given in Table 1.

Table 1. Capability example to be listed in KIO1’s system prompt.

Agent	Description	Capability values
KIO2	Bug Locate & Fix: Reverse Execution & Dynamic Slicing Component	bug_localization , diagnosis, replay, trace_alignment
...	...	...
KIO9	Responsible AI and compliance review	responsible_ai compliance_review ...
KIO10	TinyML and energy-efficiency analysis.	tinyml energy_efficiency ...
KIO11	Test automation and validation	test_automation validation ...
...	...	...

This initial KIO1 to KIO2 integration provides a controlled environment for validating the communication protocol, endpoint configuration, message structure, workflow tracking, and error handling. Once these mechanisms have been validated, the same communication model can be applied when integrating the remaining KIOs.

The progressive integration strategy therefore establishes a clear separation of responsibilities. KIO1 generates and manages the overall workflow, the Workflow Planner translates workflow steps into executable messages, and the individual KIOs execute their assigned tasks. This separation provides a common communication mechanism that can be reused as additional KIOs are progressively integrated into the platform.

2. KIO2 Capabilities

KIO1 skips any workflow step whose requested capability the target agent has not declared,

so the capability vocabulary is part of the contract rather than documentation. KIO2 declares four capabilities. All of them operate on a recorded execution of the program under analysis: KIO2 is a dynamic analysis component, and every capability therefore presupposes that the program can be executed. KIO2 capabilities are given in Table 2.

Table 2. Capabilities declared by KIO2.

Capability	Purpose	Principal output
bug_localization	Record the execution, compute a backward dynamic slice from the failure, and rank the statements on that slice by causal role.	summary, findings[], crash_state, trace_ref
diagnosis	The causal account behind the localisation, optionally contrasted with a reference (passing) run of the same program.	root_cause, evidence[], slice_context
replay	Post-mortem navigation over a recorded trace: step into, over, out, forwards and backwards; inspect recorded state and def-use.	cursor, current, timeline, def_of
trace_alignment	Compare executions of one program: distance and divergences for two traces, or a distance matrix with reference and outlier for a set.	distance, delta[], divergences[], or matrix

The capability values above are the ones to be listed for KIO2 in Table 1 and in KIO1's system prompt. Two remarks on scope. First, *fix_recommendation* is **not a KIO2 capability**: under D2.6 UC-UC1-03 the fix patch is produced by KIO7, and KIO2's contribution is the evidence for it, delivered as the *slice_context* field of the diagnosis output. A request for that capability is answered with an explicit error rather than a plausible-looking suggestion, so that a misconfiguration surfaces instead of propagating silently. Second, *replay* and *trace_alignment* are included so that the orchestrator can drive an interactive post-mortem session and compare runs, rather than being limited to a single question-and-answer exchange.

Three boundaries follow from this. KIO2 analyses an execution, so it does not examine code that has not been run and does not replace static analysis. It reports where a fault is and does not produce the corrected code ([D2.6 - Functional and NonFunctional Requirements](#)). It executes only the program it is given, calls no other agent, and fetches nothing on its own initiative: every input reaches it in the request or through data.upstream.

3. Placement of the KIO2 Step within the Workflow

In the example workflow above, the KIO2 step (s4) is scheduled immediately after code generation and before test automation. That ordering cannot be satisfied by a dynamic analysis component. At s4 no test has been generated yet, nothing has been executed, and consequently there is no failing run to analyse. The same ordering also diverges from D2.6, where UC-UC1-03 defines the sequence as KIO11 confirming the failure and KIO2 then localising it, and where every use-case chain that involves KIO2 places it after the testing stage.

The KIO2 step is therefore scheduled after the KIO11 test-automation step, and declares that dependency explicitly so that the test results are propagated to it:

```
{
  "step_id": "s8",
  "agent_id": "KIO2",
  "capability": "bug_localization",
  "depends_on": ["s7"],
  "task": "Locate the fault behind the failing tests reported by the test
          automation step, and provide the runtime evidence for a fix."
}
```

A secondary consequence follows from this correction. Because KIO1 may schedule KIO2 at a point where the program is in fact healthy, **a clean run is reported as a successful outcome**, not as an error: the reply carries status "ok" with *defect_found* set to false and an empty findings list. Reporting an absence of defects as a failure would cause every workflow over correct code to be recorded as broken.

4. Step Dependencies and Data Propagation

A workflow step may declare *depends_on*, listing the step identifiers whose results it consumes. Before dispatching a step, the Workflow Planner attaches the outputs of those steps to the execution message under *data.upstream*, keyed by step identifier. This is the mechanism by which the result of the KIO11 step reaches KIO2, and by which a trace produced by one KIO2 step reaches a later replay or comparison step. No agent-to-agent communication takes place: every exchange is mediated by the orchestrator, which keeps the topology a star and each KIO independently deployable. This is consistent with the use-case descriptions, in which KIO11 does not interact with KIO2 directly: KIO2 is invoked by KIO1, and the test-automation results reach it as upstream data. KIO2 accordingly issues no outbound calls of its own.

```
"data": {
  "upstream": {
    "s7": { "summary": "3 failing tests", "failing_tests": [ ... ] }
  }
}
```

KIO2 reads the entry point, the repository root and any trace reference out of *data.upstream* when they are not given directly, so a correctly declared dependency requires no further wiring.

5. Input Contract: the data Field for KIO2

The task field carries the natural-language instruction; data carries the structured input. KIO2 records a real execution, which imposes one requirement that the placeholder message does not yet express: **an artifact name and a repository location are not sufficient, because neither states what to run.** A runnable entry point is required. Data fields are given in Table 3.

Table 3. The data field for a KIO2 step.

Field	Required	Meaning
<code>target.entry_point</code>	Yes, for bug_localization and diagnosis	Runnable script, relative to the repository root. Accepted as <code>entrypoint</code> as well.
<code>repository.path</code>	Recommended	Repository root. Used as the working directory and as the scope for trace-target discovery. Accepted as <code>source_location</code> as well.
<code>target.functions</code>	No	Functions to instrument. When empty, KIO2 discovers them from the repository, which is the expected mode: an orchestrator knows which run failed, not which functions to instrument.
<code>failure</code>	No	Failure context: <code>test_id</code> , <code>exception_type</code> , <code>message</code> . Accepted as <code>bug_report</code> as well, as a string or an object.
<code>execution_trace</code>	For replay and trace_alignment	A trace reference previously returned by KIO2. Accepted as <code>trace_ref</code> / <code>trace_refs</code> as well.
<code>reference.trace_ref</code>	No	A passing run of the same program, used by diagnosis to report how the failing run differed.
<code>criterion</code>	No	[FILE:]LINE[:VAR], to analyse a specific value instead of the crash.
<code>source_artifact</code>	No	Recorded for traceability. KIO2 requires a path, not an artifact name.

6. Response Contract

The shared KIO1–KIO10 protocol document defines the request message but not the reply. The reply schema below is therefore proposed here, in the form already implemented by KIO2 and by the orchestrator's reference agent, so that result handling and workflow tracking can be validated together with the request path. It applies uniformly to every KIO and is independent of the capability requested.

```
{
  "workflow_id": "wf-fastapi-shifts-001",
  "step_id": "s8",
  "agent_id": "KIO2",
  "status": "ok",
  "output": { ... },
  "error": null
}
```

Reply fields are given in Table 4.

Table 4. Reply fields.

Field	Rule
workflow_id, step_id	Echoed from the request. They are how the orchestrator matches a reply to the step that produced it.
agent_id	The responding agent.
status	Either "ok" or "error". Always set explicitly: an absent status is interpreted as success, so omitting it would record a failed step as a successful one with no result.
output	A JSON object whose shape is defined by the responding KIO. The orchestrator stores it verbatim and propagates it to dependent steps. Ignored when status is "error".
error	A short diagnostic message when status is "error".

The output object for the bug_localization capability follows the structure below. Source locations are reported relative to the repository root: absolute paths inside the KIO2 container carry no meaning for the orchestrator and would expose the deployment layout. The per-finding confidence combines the confidence of the run with the evidence weight of the statement, so that the leading finding carries the confidence of the analysis as a whole.

```
{
  "summary": "Located the fault at shop/pricing.py:11 (discounted).",
  "verdict": "defect",
  "defect_found": true,
  "findings": [
    {
      "file": "shop/pricing.py", "function": "discounted", "line": 11,
      "confidence": 0.85, "score": 1.0, "dependency": "criterion",
      "description": "rate = {\"gold\": 0.20, \"silver\": 0.10}[tier]"
    }
  ],
  "crash_state": { "net": "230", "tier": "'bronze'" },
  "criterion": "exception@shop/pricing.py:11",
  "slice_size": 6,
  "confidence": 0.85,
  "trace_ref": "kio2://trace/QzpcVXNlcnN...",
  "hitl_required": false
}
```

The diagnosis capability returns *root_cause*, an evidence list, and *slice_context*, the value-annotated program slice intended for the fix generation step. The *root_cause* statement is

derived from the recording rather than inferred by a language model: it names the statement that failed, the values that were in scope, and the statements that produced those values. When a reference run is supplied, the evidence list additionally reports the points at which the two executions diverged and the variables whose recorded values differ.

7. Machine-Readable Contract

The request and reply structures above are additionally published as JSON Schema, so that the orchestrator can validate against them rather than reproducing the shapes by hand. They are retrievable from the running service, which makes the contract discoverable at integration time rather than dependent on a document version. API endpoints are given in Table 5.

Table 5. Contract discovery.

Endpoint	Returns
GET /schema	Both schemas, keyed request and reply.
GET /schema?name=response	A single schema.
GET /tasks	The declared capabilities and, in <code>schema_url</code> , where the schemas are served from.

The reply schema is **enforced rather than descriptive**: KIO2's test suite validates an actual reply from each capability, including both error paths, against it, so the schema cannot diverge from the implementation without a test failing. The request schema is published for the caller's benefit and is deliberately **not** applied to inbound messages: KIO2 accepts the field-name aliases listed in Table 3 and inherits missing values from `data.upstream`, so an evolution of the protocol on the orchestrator's side does not render KIO2 unavailable.

One field in the reply deserves particular attention. Every localization and diagnosis result carries **verdict**, taking the value "defect" when statements were implicated, "clean" when the target ran to completion with nothing to localise, and "inconclusive" when the run was analysed but no statement could be attributed. This exists so that a consumer need not infer an answer from the length of the findings list, and, more importantly so that **no consumer needs to know where KIO2 is positioned in the workflow**. A deployment step reads it to decide whether to proceed, a fix-generation step reads it to decide whether to run, and the orchestrator surfaces it directly. The ordering of steps therefore remains a workflow concern rather than a matter of interface compatibility.

8. Transport Rules

The rules below apply to every message, whatever capability is requested. They are stated together because a caller implements them once rather than rediscovering them per capability. Rules are given in Table 6.

Table 6. Transport rules.

Rule	Value
Media type	Requests and replies use application/json with UTF-8 encoding. No other media type is accepted on /execute.
HTTP status codes	200 for every message KIO2 was able to read, whether the task succeeded or failed. 422 when the body is not valid JSON. 404 for an unknown path or an unknown schema name. The service produces no other code.
Path form	Every path in a reply is POSIX separated and relative to the repository root. An absolute path from inside the container has no meaning for the orchestrator and would expose the deployment layout.
Identifiers	workflow_id, step_id and agent_id are opaque strings and are echoed unchanged. KIO2 generates none of them.
Trace references	A recording is addressed as kio2://trace/<token>. It resolves only under the configured trace directory, and a reference pointing outside it is refused.
Field names	The alternative names listed in Table 3 are accepted alongside the canonical ones, and a value that is absent is inherited from data.upstream. An inbound message is therefore not rejected on schema grounds.
Recording budget	45 seconds for a step dispatched over this protocol, so that the reply arrives inside the 60 second dispatch timeout. KIO2's own contract allows 180 seconds for the same work, which a dispatched step cannot use today. The budget is set with KIO2_TRACE_BUDGET.
Concurrency	KIO2 keeps no state between requests and is safe under concurrent ones. Each localisation starts a subprocess that runs the target program, so the number of analyses in parallel is bounded by the resources of the deployment rather than by the protocol.
Retry	A failed task is deterministic today: the same request produces the same failure, so repeating it unchanged does not help. A retry is useful only once the cause named in the error has been removed.

9. Status, Error and Timing Semantics

Four conventions govern the exchange and are stated here because they determine how a KIO must be implemented rather than merely how it is called. Conventions are given in Table 7

Table 7. Interaction conventions.

Convention	Rationale
A failed task is reported as HTTP 200 with status "error".	A non-2xx response is interpreted as the service being unavailable, which is a different condition from the task having failed, and is handled differently by the orchestrator.
Dispatch timeout of 60 seconds, without retry.	For the current KIO1–KIO2 integration prototype, the dispatch timeout is 60 seconds and no automatic retry is performed. KIO2 accordingly bounds recording to 45 seconds, leaving room for analysis and the reply, and returns a diagnostic error rather than being abandoned mid-execution.
Steps may be dispatched concurrently.	Independent steps of a plan are executed in parallel, so an agent may receive several simultaneous requests and must be safe under concurrency.
Requests carry a W3C traceparent header.	Adopting it places the agent's spans in the same distributed trace as the orchestrator's, so that one workflow appears as a single trace in the shared observability stack.

The status field says whether the task was carried out. When it is "error", the error field explains why, and today it carries a free text message. A caller that must react differently to different failures has to read that text, which is not a contract. The vocabulary below is therefore proposed for the next KIO2 release, to be reported alongside the existing message so that a failure can be classified without parsing prose. Until it is implemented, only status is contractual. Error codes are given in Table 8.

Table 8. Proposed error codes.

Code	Retryable	Meaning
malformed_request	no	The message could not be read as an execution message, or a field carried a value outside its permitted set.
unsupported_capability	no	The requested capability is not one of the four in Table 2. A request for fix_recommendation is answered with this code.
entry_point_missing	no	No runnable entry point was supplied and none could be found in data.upstream. This is the most frequent integration error.
artifact_unavailable	once	The repository root or the entry point could not be read at the location given.
trace_unavailable	once	The referenced recording could not be resolved, or could not be

		read.
trace_ref_refused	no	The reference points outside the configured trace directory.
analysis_failed	no	The program ran but the execution could not be recorded, or the recording could not be sliced.
deadline_exceeded	with a larger budget	Recording did not finish within the budget of Table 6.
service_unavailable	yes	The service is starting up or shutting down.

10. Long Analyses

KIO2 executes the program under analysis, so the time it needs is the time that program needs. The present exchange is synchronous and is bounded twice: the orchestrator waits 60 seconds and does not retry, and KIO2 stops recording after 45 seconds so that it can still answer. A small reproduction script fits in that window. The test suite of a real repository often does not, and KIO2's own contract already allows 180 seconds for the same work. Nothing in the current protocol carries that case, so a long analysis is reported as a failure although the component is working correctly. The proposal is the pattern the KIO1 to KIO9 and KIO1 to KIO12 protocols already define, so that a caller implements one mechanism for all three agents: an analysis that does not fit the synchronous budget is accepted with HTTP 202 and a job identifier, and the caller collects the result when it is ready. This is proposed and is not implemented in KIO2 today. Details are given in Table 9.

Table 9. Proposed choice of processing mode.

Condition	Behaviour
The estimated duration fits the synchronous budget	Processed at once. HTTP 200 with the complete reply, as today.
The caller asks for asynchronous handling	Accepted for later processing. HTTP 202 with a job identifier.
The estimated duration exceeds the synchronous budget	Accepted for later processing. HTTP 202 with a job identifier.
The caller requires a synchronous answer and the duration exceeds the budget	HTTP 200 with status "error", stating that the deadline cannot be met.

HTTP/1.1 202 Accepted

{

```

"workflow_id": "wf-fastapi-shifts-001",
"step_id": "s8",
"agent_id": "KIO2",
"capability": "bug_localization",
"status": "accepted",
"job": {
  "job_id": "job-3c9d11",
  "status_uri": "/jobs/job-3c9d11",
  "result_uri": "/jobs/job-3c9d11/result",
  "retry_after_s": 15
},
"output": null,
"error": null
}

```

Two endpoints complete the pattern. GET `/jobs/{job_id}` reports the state of the analysis and how long to wait before asking again, and GET `/jobs/{job_id}/result` returns the ordinary reply once the analysis has finished. A job that expired is not a step that passed.

11. Execution Traces and Their Lifecycle

The *execution_trace* field appears in the request as an input, and its provenance requires definition. Traces are produced by KIO2 itself: a *bug_localization* or *diagnosis* step records the target program and returns a **trace reference** in its output. That reference is what a subsequent *replay* or *trace_alignment* step consumes, whether it is supplied directly or inherited through *data.upstream*.

A trace reference is an opaque handle of the form `kio2://trace/<token>` rather than a filesystem path, so that no detail of the KIO2 deployment crosses the service boundary. It is stateless: any KIO2 instance with access to the same trace directory resolves the same reference, which permits replication. References resolve only within the configured trace directory; a reference pointing outside it is refused, since the orchestrator interface is a network boundary.

Two operational consequences should be recorded at platform level. Traces are retained after a step completes, because the replay and comparison capabilities depend on them; a retention policy is therefore required for continuous operation. And because a trace reference is resolvable only where the trace resides, a deployment that replicates KIO2 must either share the trace directory between replicas or route successive steps of one workflow to the same instance.

12. Recorded Values and Data Protection

A KIO2 reply carries values that were actually in memory while the program ran. crash_state holds the variables in scope at the failure, a replay view holds the recorded state at each step, and the diagnosis evidence quotes those values directly. The recording holds the same values for the whole run. This is what makes the result evidence rather than a guess, and it is also what makes it sensitive: if the program under analysis processes personal data, the recorded values contain personal data. KIO2 does not mask values and does not delete recordings. Both are therefore obligations of the deployment, and they are listed below so that they are assigned rather than assumed. Details are given in Table 10.

Table 10. Handling of recorded values.

Obligation	Owner
Reproduce the failure with synthetic or pseudonymised data wherever it can be reproduced that way.	The step that prepares the input, normally the test automation step.
Treat the trace directory as sensitive storage, with restricted access and no shared volume beyond the deployment.	The deployment.
Define how long a recording is kept and delete it when that period ends.	The deployment. A recording is needed only while a replay or comparison step may still refer to it.
Record which workflow and step produced a recording, so that a deletion request can be answered.	The orchestrator.
Decide whether the recorded values of a given application may be stored at all.	The compliance review. This is a KIO9 question rather than a KIO2 one.

13. Service Endpoint, Health and Capability Discovery

KIO2 is a containerised HTTP service without a user interface of its own, consistent with the platform model in which KIO1 owns the graphical and command-line interfaces. The container listens on **port 8102**, the value already registered for KIO2 in the orchestrator's dispatch configuration and consistent with its per-agent scheme. The host part of the endpoint depends on the deployment topology: http://KIO2_IP:8102 on a single machine, or <http://kio2:8102> where the service name resolves. The listening port remains configurable. Endpoints are given in Table 11.

Table 11. Endpoints exposed by KIO2.

Endpoint	Method	Purpose
/execute	POST	Accepts an execution message; returns a reply.
/health	GET	Liveness probe. Returns the agent identifier and status.
/tasks	GET	Capability discovery. Returns the capabilities the service answers for, and those it explicitly declines.
/schema	GET	Contract discovery. Returns the published request and response schemas; a name parameter selects one of them.

Capability discovery is worth using during integration rather than relying on the registered configuration alone: a step requesting a capability that the agent has not declared is skipped rather than dispatched, and querying the service directly makes such a mismatch immediately visible.

14. Deployment and Configuration

KIO2 is delivered as a container image and is configured entirely through environment variables. The repository under analysis has to be reachable inside the container, normally as a mounted volume whose path inside the container is the one passed as the repository root. The bundled example needs no volume and is the quickest way to confirm that the service and the engine are wired correctly. Configurations are given in Table 12.

Table 12. Configuration.

Variable	Default	Purpose
KIO_PORT	8102 in the image	Listening port. 8102 is the value registered for KIO2 in the dispatch configuration of the orchestrator.
KIO_HOST	0.0.0.0 in the image	Listening interface.
KIO2_TRACE_BUDGET	45	Longest recording, in seconds, for a step dispatched over this protocol. Raise it together with the dispatch timeout, never alone.
KIO2_TRACE_DIR	System temporary directory	Where recordings are written. It is also the only directory a trace reference resolves under, and the directory replicas have to share.

A replicated deployment has one further condition. A trace reference resolves only where the recording is, so either every replica shares the same trace directory, or the successive steps of one workflow are routed to the same instance. Sizing follows from the same fact: every localisation starts a subprocess that runs a whole program, so one replica serves few analyses at a time rather than many.

15. Deployment Profile and Authentication

KIO2 exposes no authentication and no authorisation today. For a service that executes the code it is given, this deserves a plain statement rather than silence: an unprotected /execute endpoint accepts a repository path and an entry point from anyone who can reach it, and runs them. Being on an internal network is a deployment fact, not an authentication mechanism.

The two profiles below carry the same names as the KIO1 to KIO12 protocol, so that one deployment decision covers both agents. The current KIO2 build satisfies profile P. Profile R requires work on the KIO2 side and is listed among the open items. Deployment profiles are given in Table 13.

Table 13. Deployment profiles.

Concern	Profile P, prototype	Profile R, production
Permitted input	Sample repositories, and reproductions built from synthetic or pseudonymised data.	Real repositories, subject to the rows below and to Table 10.
Transport security	TLS recommended. Plain HTTP acceptable on an internal network.	TLS required on every endpoint except /health.
Authentication	Not required.	Required. A bearer token or mutual TLS issued by the deployment.
Authorisation	Not enforced.	Required, per capability, so that a caller allowed to localise is not thereby allowed to replay the recordings of other workflows.
Isolation	The container runs without credentials in its environment and without write access beyond the repository under analysis.	As profile P, and with outbound network access restricted, since the code being executed is not KIO2's own.

16. How Each Capability Is Used

The table below is the summary an orchestrator needs: when to call each capability, what to send, what to read back, and what the answer means for the workflow. Interactions are given in Table 14.

Table 14. KIO2 interactions.

Capability	Trigger	Request carries	Reply read	Effect on the workflow
bug_localization	A test step reported a failure.	The entry point and the repository root,	verdict, findings, trace_ref	defect continues to the fix step.

		and the failure context when it is known.		clean is a successful step with nothing to do. inconclusive needs review.
diagnosis	A fault was located and the fix step needs the evidence.	The same input, plus the trace reference of a passing run when one exists.	root_cause, evidence, slice_context	slice_context is passed to the fix generation step. No transition of its own.
replay	A person or the interface walks through the recorded failure.	A trace reference and a position in the recording.	current, timeline, def_of	None. The step is read only and re-executes nothing. The returned cursor is sent back on the next call.
trace_alignment	Two or more runs of the same program have to be compared.	Two or more trace references.	distance and delta for a pair; matrix, reference and outlier for a set.	None by itself. In a set, the outlier is the run that behaved differently.

17. Versioning and Compatibility

The rules below state what a caller may rely on between versions of this document. Details is given in Table 15.

Table 15. Compatibility guarantees.

Guarantee	Statement
Field meanings do not change	No field named in version 0.2 changes its meaning or its type in version 0.3.
Additions are additive	A caller written against version 0.2 receives a valid reply from a version 0.3 service. New fields may be ignored.
Closed sets stay closed	status is ok or error. verdict is defect, clean or inconclusive. dependency is criterion, control or data. An alignment is pair or set. A caller meeting an unrecognised value treats it as one it cannot act on and asks for review.
Alternative field names are kept	The names listed in Table 3 remain accepted. They are not removed within this version line.
The reply schema is enforced	KIO2's tests validate a real reply from every capability against the published schema, so the schema cannot drift from the service without a test failing.
Proposals are marked	What this document marks as proposed changes nothing for a caller until it is implemented and announced in /tasks.

18. Constraints and Open Decisions

The following items constrain the integration and are recorded here so that they are resolved deliberately rather than encountered during deployment. Decision requiring items are given in Table 16.

Table 16. Constraints and items requiring a decision.

Item	Status
Language scope	KIO2 supports Python only. FR-KIO2-01 permits Python and/or C; the proof-of-concept selected Python, and the tracing engine instruments a Python runtime. The UC3 use-case (FPGA, C++/HLS) is consequently outside the current scope of KIO2 and requires either a decision to extend the language support or a revision of that use-case's KIO2 requirements.
Ownership of entry-point resolution	D2.6 specifies KIO2's input as failing test results or runtime logs, whereas KIO2 requires a runnable entry point. The translation between the two is currently unassigned. The recommended allocation is to the orchestration layer, which can convert a failing test identifier into a reproduction command, so that KIO2 remains a pure execution-analysis service.
Role of KIO2 relative to KIO7	D2.6 is not internally consistent. UC-UC1-03 assigns fault localisation to KIO2 and fix generation to KIO7, which is the reading implemented here; four other passages, and every use-case chain, assign fixing or verification duties to KIO2 and place it after KIO7. The direction must be agreed before the orchestrator is wired, since it determines whether KIO2 is invoked before or after code generation.
Execution of the code under analysis	Obtaining a runtime trace requires executing the target program. The KIO2 container therefore executes code originating from the repository under analysis and should be deployed as a sandbox: without credentials in its environment, without write access to shared volumes beyond the analysed repository, and with restricted network egress.
Trace retention	Recorded traces are retained so that replay and comparison steps can use them. No retention policy is defined; one is required before continuous operation.